

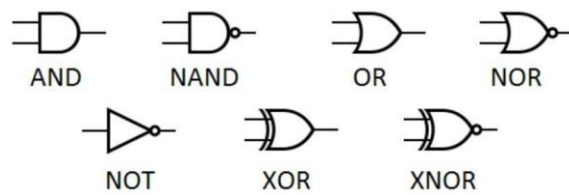
Assignment No.

01

Combinational

Logic Design

Q1. Design and Simulate all the six fundamental logic gates OR, NOT, NAND, NOR, XOR, XNOR in Vivado/ Modelsim in dataflow and structural modeling styles.



CODE:

```

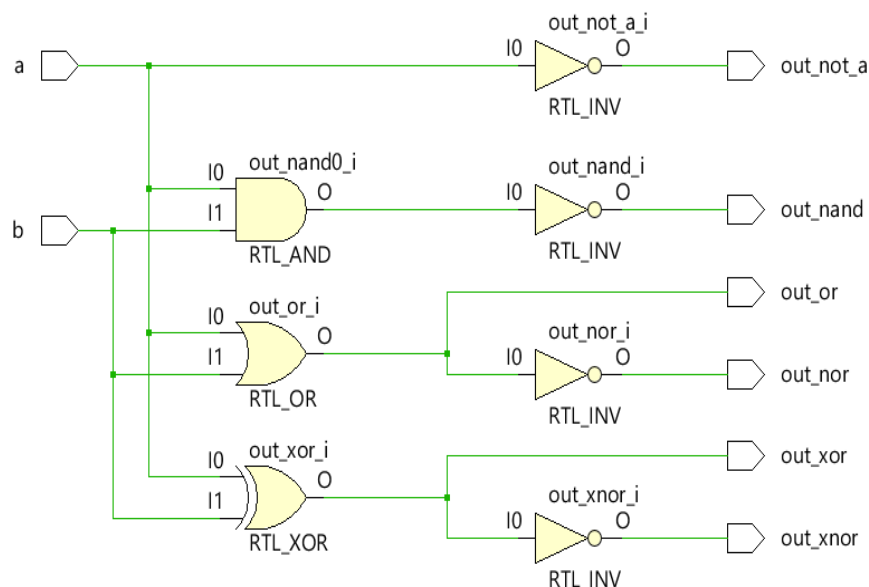
module all_gates_dataflow(
    input a,
    input b,
    output out_not,
    output out_or,
    output out_nand,
    output out_nor,
    output out_xor,
    output out_xnor);

    assign out_not = ~a;
    assign out_or = a | b;
    assign out_nand = ~(a & b);
    assign out_nor = ~(a | b);
    assign out_xor = a ^ b;
    assign out_xnor = ~(a ^ b);

endmodule

```

SCHEMATIC:



TESTBENCH CODE:

```

module tb_all_gates;

    reg a;
    reg b;

    wire df_not_a, df_or, df_nand, df_nor, df_xor, df_xnor;

    all_gates_dataflow uut_df (
        .a(a), .b(b),
        .out_not_a(df_not_a), .out_or(df_or), .out_nand(df_nand),
        .out_nor(df_nor), .out_xor(df_xor), .out_xnor(df_xnor));

    initial begin

        $display("Time\t A B | NOT OR  NAND NOR XOR XNOR");
        $monitor("%0t\t %b %b |  %b  %b   %b   %b  %b   %b",
            $time, a, b, df_not_a, df_or, df_nand, df_nor, df_xor,
            df_xnor);

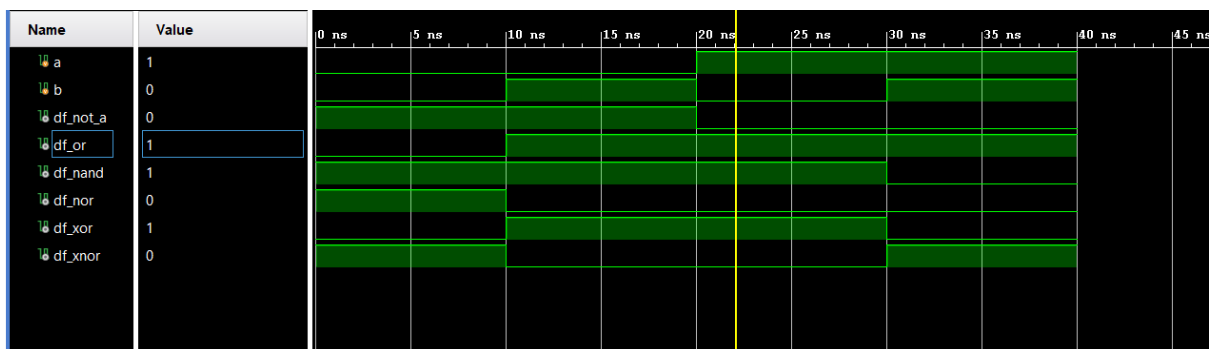
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        $finish;
    end

endmodule

```

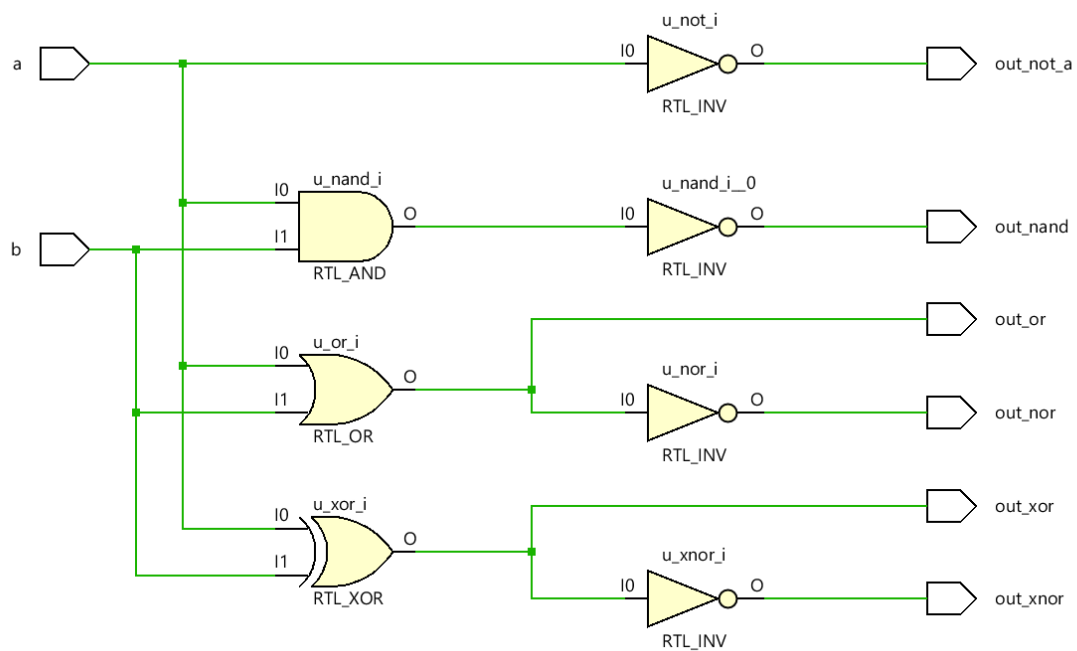
SIMULATION:



CODE:

```
module all_gates_structural(  
    input a,  
    input b,  
    output out_not_a,  
    output out_or,  
    output out_nand,  
    output out_nor,  
    output out_xor,  
    output out_xnor);  
  
    not u_not (out_not_a, a);  
    or u_or (out_or, a, b);  
    nand u_nand (out_nand, a, b);  
    nor u_nor (out_nor, a, b);  
    xor u_xor (out_xor, a, b);  
    xnor u_xnor (out_xnor, a, b);  
  
endmodule
```

SCHEMATIC:



TESTBENCH CODE:

```
module tb_all_gates_structural;

    reg a;
    reg b;

    wire str_not_a, str_or, str_nand, str_nor, str_xor, str_xnor;

    all_gates_structural uut_str (
        .a(a), .b(b),
        .out_not_a(str_not_a), .out_or(str_or), .out_nand(str_nand),
        .out_nor(str_nor), .out_xor(str_xor), .out_xnor(str_xnor)
    );

    initial begin

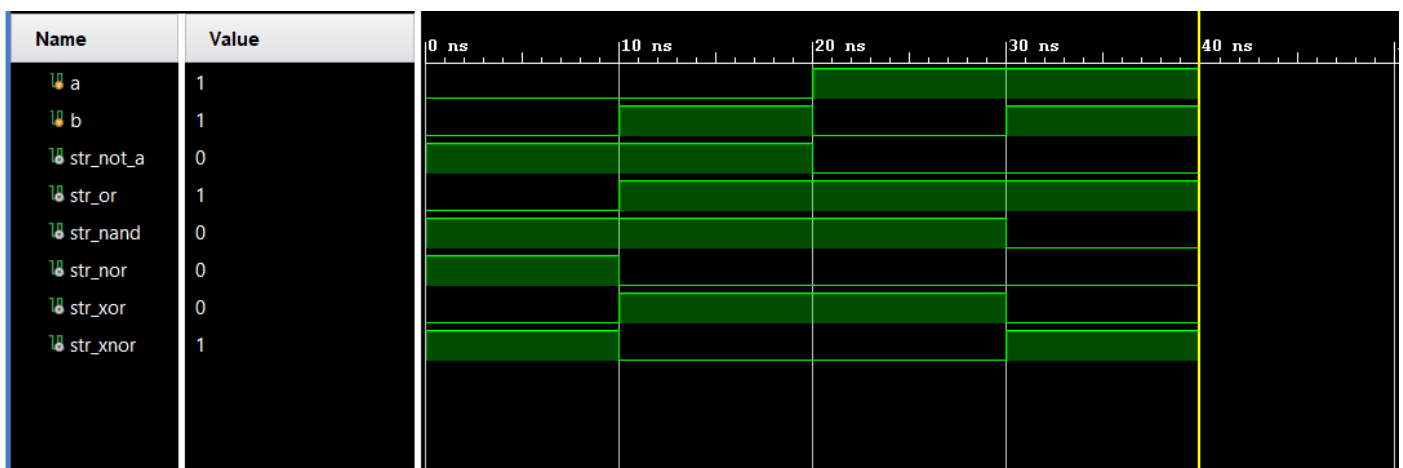
        $display("Time\t A B | NOT OR  NAND NOR XOR XNOR");
        $monitor("%0t\t %b %b | %b  %b  %b  %b  %b  %b",
            $time, a, b, str_not_a, str_or, str_nand, str_nor, str_xor,
str_xnor);

        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        $finish;
    end

endmodule
```

SIMULATION:



Q2. Design a 2:1 Multiplexer using Verilog HDL in different modeling styles.

a) Dataflow Modeling using continuous assignment (assign) statement.

b) Dataflow Modeling using the conditional (?:) operator.

c) Behavioral Modeling using always block.

d) Behavioral Modeling using if-else statements.

e) Behavioral Modeling using case statements.

f) Gate-Level Modeling using logic primitives (and, or, not).

Answer:

2:1 MUX takes two data inputs (I_0, I_1), a single select line (S), and routes one of the inputs to the output (Y). If $S = 0, Y = I_0$. If $S = 1, Y = I_1$.

a) Dataflow Modeling using continuous assignment statement.

```
module mux21_dataflow_assign (  
    input i0,  
    input i1,  
    input s,  
    output y);  
  
    assign y = (i0 & ~s) | (i1 & s);  
  
endmodule
```

b) Dataflow Modeling using the conditional (?:) operator.

```
module mux21_dataflow_conditional (  
    input i0,  
    input i1,  
    input s,  
    output y);  
  
    assign y = s ? i1 : i0;  
  
endmodule
```

c) Behavioral Modeling using always block.

```
module mux21_behavioral_always (  
    input i0,  
    input i1,  
    input s,  
    output reg y);  
  
    always @(*) begin  
        y = (i0 & ~s) | (i1 & s);  
    end  
endmodule
```

d) Behavioral Modeling using if-else statements.

```
module mux21_behavioral_ifelse (  
    input i0,  
    input i1,  
    input s,  
    output reg y);  
  
    always @(*) begin  
        if (s == 1'b1) begin  
            y = i1;  
        end else begin  
            y = i0;  
        end  
    end  
endmodule
```

e) Behavioral Modeling using case statements.

```
module mux21_behavioral_case (  
    input i0,  
    input i1,  
    input s,  
    output reg y);  
  
    always @(*) begin  
        case (s)  
            1'b0:    y = i0;  
            1'b1:    y = i1;  
            default: y = i0;  
        endcase  
    end  
endmodule
```

endmodule

f) Gate-Level Modeling using logic primitives (and, or, not).

```
module mux21_gate_level (  
    input i0,  
    input i1,  
    input s,  
    output y  
);  
  
    wire s_bar;  
    wire w0;  
    wire w1;  
  
    not u_not (s_bar, s);  
    and u_and0 (w0, i0, s_bar);  
    and u_and1 (w1, i1, s);  
    or u_or0 (y, w0, w1);  
  
endmodule
```